

# ESTRUCTURAS DE CONTROL

Apunte de Cátedra

*Algoritmos y Estructuras de Datos*

*Ingeniería en Sistemas de Información*

*Universidad Tecnológica Nacional – Facultad Regional Rosario*

## 1. Introducción

El presente apunte corresponde a la unidad Estructuras de Control de la asignatura Algoritmos y Estructuras de Datos, perteneciente al primer año de la carrera Ingeniería en Sistemas de Información de la Facultad Regional Rosario – Universidad Tecnológica Nacional.

Esta unidad tiene como objetivo que el estudiante:

- Comprenda el concepto de control del flujo de ejecución de un algoritmo.
- Identifique y utilice correctamente las estructuras secuenciales, condicionales y repetitivas.
- Exprese soluciones algorítmicas mediante diagramas de Nassi-Shneiderman (chapin) y las implemente en un lenguaje de programación (Python).
- Desarrolle algoritmos correctos, legibles y verificables, respetando el paradigma de programación estructurada.

El tratamiento del tema se apoya en el Teorema de Böhm y Jacopini, que establece que **todo algoritmo puede construirse utilizando únicamente estructuras de secuencia, selección e iteración.**

## 2. Conceptos Fundamentales

### 2.1. ¿Qué son las Estructuras de Control?

Las **estructuras de control** son construcciones que dirigen el flujo de ejecución de un programa. Permiten que un algoritmo tome decisiones, repita acciones y organice la secuencia en que se ejecutan las instrucciones.

Sin estructuras de control, un programa ejecutaría todas sus instrucciones de forma lineal, una tras otra, sin posibilidad de adaptarse a diferentes situaciones o de repetir operaciones.

Las estructuras de control son la base de la lógica algorítmica y constituyen los pilares fundamentales de la programación estructurada.

## 2.2. Tipos de Estructuras de Control

Según el Teorema de Böhm y Jacopini (1966), todo algoritmo puede expresarse mediante tres tipos básicos de estructuras:

1. **Estructuras secuenciales:** Las instrucciones se ejecutan una tras otra, en el orden en que aparecen.
2. **Estructuras selectivas (condicionales):** Permiten elegir entre diferentes caminos de ejecución según se cumpla o no una condición.
3. **Estructuras iterativas (repetitivas):** Permiten ejecutar un bloque de instrucciones varias veces.

Este teorema demuestra que no se necesitan otras estructuras para expresar cualquier algoritmo computable, lo que constituye el fundamento teórico de la programación estructurada.

## 2.3. Diagramas de Nassi-Shneiderman (chapin) y Python

A lo largo de este apunte se describirán conceptualmente los diagramas chapin correspondientes a cada estructura, permitiendo al estudiante visualizar la lógica de control de manera clara y estructurada.

Se acompañará también cada estructura de control con ejemplos en pseudocódigo y en lenguaje de programación Python.

## 3. Estructura Secuencial

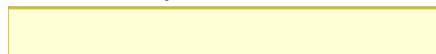
### 3.1. Concepto

La estructura secuencial es la forma más básica de control. Las instrucciones se ejecutan **una a continuación de la otra**, en el orden exacto en que aparecen escritas en el código, sin desvíos ni repeticiones.

Esta estructura representa el flujo natural de ejecución de un programa: cada instrucción se completa antes de pasar a la siguiente. Es la estructura fundamental sobre la cual se construyen las demás.

### 3.2. Diagrama de Nassi-Shneiderman

En un chapin, la estructura secuencial se representa como un único bloque rectangular



que contiene las instrucciones ordenadas verticalmente. No

presenta divisiones internas ni bifurcaciones. Cada instrucción ocupa una fila del bloque, ejecutándose de arriba hacia abajo.

### 3.3. Ejemplo Práctico

#### Problema: Calcular el área de un rectángulo

#### Pseudocódigo:

```
Algoritmo AreaRectangulo
    Definir base, altura, area Como Real
    Leer base
    Leer altura
    area <- base * altura
    Mostrar "El área es:", area
FinAlgoritmo
```

#### Chapin:

|                                |
|--------------------------------|
| Leer (altura)                  |
| Leer (base)                    |
| area ← base * altura           |
| Mostrar ("El area es: ", area) |

VAR

base, altura, area: enteros

**Explicación:** En este algoritmo, cada instrucción se ejecuta en secuencia: primero se solicita la altura, luego la base (esto primeros dos pasos pueden ser en orden indistinto), luego se calcula el área y finalmente se muestra el resultado. El cálculo del área debe ser si o si después de los ingresos de datos (de los Leer). Y el Mostrar el resultado si o si después del cálculo. Acá no hay decisiones ni repeticiones.

#### Alternativa:

Una forma abreviada del algoritmo anterior podría ser:

|                                         |
|-----------------------------------------|
| Leer (altura)                           |
| Leer (base)                             |
| Mostrar ("El area es: ", base * altura) |

VAR

base, altura: enteros

Esto es conveniente realizarlo siempre y cuando la variable **area** no se utilice más adelante en el algoritmo (acá nos ahorramos un paso y una variable respect del algoritmo anterior).

### Aclaraciones:

- 1) En los diagramas de chapin evitamos los mensajes aclaratorios sobre el ingreso de datos (por ej. Mostrar "Ingrese la base") pero si son obligatorios los mensajes con la salida de información del algoritmo (por ej. Mostrar "El area es: ", area). El objetivo es centrarnos en la esencia del algoritmo (qué es lo que hay que hacer para resolver el problema). Luego, en los programas en Python podemos incluir todos los detalles necesarios.
- 2) Las funciones de salida de datos (**Mostrar, Exhibir o Escribir**) son equivalentes y válidas.
- 3) En los diagramas de chapin colocamos una sección de declaración de variables, fuera del mismo, para definir las variables que vamos a usar y de que tipo de datos son (Por ej.: VAR base, altura, area: enteros)
- 4) En la cátedra utilizaremos los diagramas de chapin con su declarativa de variables para resolver los algoritmos.

### Implementación en Python:

```
base = float(input("Ingrese la base: "))
altura = float(input("Ingrese la altura: "))
area = base * altura
print("El área es:", area)
```

**Observación:** la función *float()* que encierra al input es necesaria para convertir el ingreso de tipo string que tiene por defecto el input de Python a número real, ya que luego, tanto la base como la altura son dos variables con las que realizamos operaciones aritméticas.

Si utilizamos *int()* convertimos el ingreso a número entero.

## 4. Estructuras Condicionales (Selectivas)

### 4.1. Concepto

Las estructuras condicionales permiten que un algoritmo (y su posterior programa) **tome decisiones**. La ejecución de uno u otro conjunto de instrucciones depende de la evaluación de una o más condiciones lógicas (expresiones booleanas que resultan verdaderas o falsas).

Estas estructuras son fundamentales para crear programas que se adapten a diferentes situaciones y datos de entrada, permitiendo que el flujo de ejecución varíe según las circunstancias.

### 4.2. Condicional Simple (if)

#### Descripción

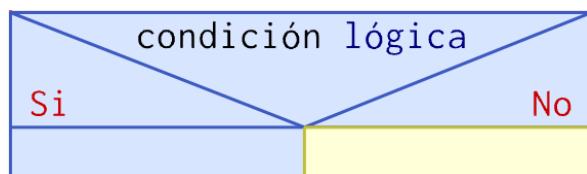
La estructura condicional simple ejecuta un bloque de instrucciones únicamente si una condición lógica o booleana es verdadera. Si la condición es falsa, el bloque se omite y el programa continúa con la siguiente instrucción después del condicional.

#### Sintaxis general:

```
Si condicion Entonces
    instruccion1
    instruccion2
    ...
FinSi
```

#### Diagrama de Nassi-Shneiderman

Un bloque principal que contiene un subbloque condicionado, identificado por una expresión o condición lógica en la parte superior. El subbloque de instrucciones solo se ejecuta si la condición es verdadera. Si es falsa, el bloque queda vacío (no se ejecuta nada).



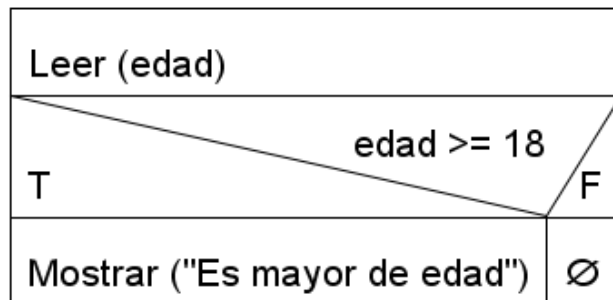
#### Ejemplo Práctico

**Problema:** Verificar si una persona es mayor de edad.

## Pseudocódigo:

```
Algoritmo MayorDeEdad
    Definir edad Como Entero
    Leer edad
    Si edad >= 18 Entonces
        Mostrar "Es mayor de edad"
    FinSi
FinAlgoritmo
```

## Chapin:



VAR

edad: entero

Observar que solo está programada la parte del **Si** o del **Verdadero (True)** luego de evaluar la condición lógica, es decir, la parte del **No** o **Falso (False)** en este caso queda vacía.

## Implementación en Python:

```
edad = int(input("Ingrese su edad: "))
if edad >= 18:
    print("Es mayor de edad")
```

**Nota:** Si la edad es menor a 18, no se muestra ningún mensaje.

**Antes de continuar con las estructuras de control, veremos un tema muy importante, que aparecen tanto en las estructuras de control de decision y en algunas de iteración. Estos son los Operadores Relacionales (o de Comparación) y los Operadores Lógicos.**

## Operadores Relacionales y Lógicos

Como vimos en el ejemplo anterior, apareció una expresión de la forma **edad >= 18**, esta expresión se la denomina **expresión lógica o booleana**, y su resultado puede ser **verdadero o falso**. El **>=** es un **operador relacional**, hay otros más que se explican a continuación.

### Operadores Relacionales

Los **operadores relacionales** permiten comparar valores y establecer relaciones de orden entre variables. Estos operadores devuelven un valor booleano (verdadero o falso).

| Operador     | Significado       | Ejemplo                |
|--------------|-------------------|------------------------|
| <b>==</b>    | Igual a           | <code>x == 5</code>    |
| <b>!=</b>    | Distinto de       | <code>x != 5</code>    |
| <b>&gt;</b>  | Mayor que         | <code>x &gt; 5</code>  |
| <b>&lt;</b>  | Menor que         | <code>x &lt; 5</code>  |
| <b>&gt;=</b> | Mayor o igual que | <code>x &gt;= 5</code> |
| <b>&lt;=</b> | Menor o igual que | <code>x &lt;= 5</code> |

Los **operandos** son los términos que aparecen a izquierda y derecha del operador (en la tabla serían la variable **x** y la constante **5**). Estos operandos pueden ser variables, constantes o expresiones, como por ejemplo `x > y+1`.

Es importante que los operandos sean del mismo tipo de datos.

### Operadores Lógicos

Los operadores lógicos permiten combinar expresiones lógicas o booleanas para formar condiciones más complejas:

- **and (Y lógico)**: Devuelve verdadero solo si ambas expresiones son verdaderas.
- **or (O lógico)**: Devuelve verdadero si al menos una de las expresiones es verdadera.
- **not (Negación)**: Invierte el valor de verdad de una expresión.

#### Ejemplo:

```
edad = 25
tiene_licencia = True
# Puede conducir si es mayor de 18 Y tiene licencia
if edad >= 18 and tiene_licencia:
    print("Puede conducir")
else:
    print("No puede conducir")
edad = 25
tiene_licencia = False
```

```
# Puede conducir si es mayor de 18 Y tiene licencia
if edad >= 18 and tiene_licencia:
    print("Puede conducir")
else:
    print("No puede conducir")

# para probar, ir cambiando el valor de la variable edad en el código
# y el de la variable tiene_licencia con True o False e ir ejecutandolo
# Luego, como ejercicio, pueden agregar inputs a edad y tiene_licencia
# pueden pedir el ingreso de un "Si" o un "No" a una variable para ver si
# tiene licencia, y luego con un if asignarle True o False a tiene_licencia.
```

## Tabla de verdad

Una tabla de verdad es una representación sistemática de todos los posibles resultados de una expresión lógica, mostrando qué valor (Verdadero o Falso) produce la expresión para cada combinación posible de valores de sus variables.

Es como una "tabla de multiplicar" pero para la lógica booleana.

## Operadores Lógicos Básicos

### 1. AND (Y lógico)

Solo es verdadero cuando **ambas** condiciones son verdaderas.

Supongamos que **A** es una expresión lógica (por ejemplo  $x > 5$ ) y **B** es otra expresión lógica (por ejemplo  $x > y + 1$ ). Entonces, al evaluar ambas expresiones lógicas con un operador lógico (como por ejemplo  $x > 5$  Y  $x > y + 1$ ) tenemos las siguientes situaciones (en la tabla siguiente las expresamos como **A and B** que es lo mismo que **A Y B**):

| A | B | A and B |
|---|---|---------|
| V | V | V       |
| V | F | F       |
| F | V | F       |
| F | F | F       |

## 2. OR (O lógico)

Es verdadero cuando **al menos una** condición es verdadera. Podemos usar (**A O B**)

| A | B | A or B |
|---|---|--------|
| V | V | V      |
| V | F | V      |
| F | V | V      |
| F | F | F      |

## 3. NOT (Negación)

Invierte el valor de verdad.

| A | not A |
|---|-------|
| V | F     |
| F | V     |

Continuando con las estructuras de control, habíamos visto el **Condicional Simple**, ahora sigue el **Condicional Doble o Completo**.

### 4.3. Condicional Doble o Completa (if-else)

#### Descripción

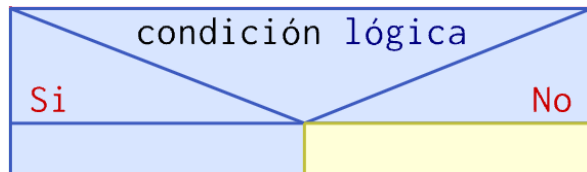
La estructura condicional doble permite seleccionar entre dos alternativas mutuamente excluyentes. Si la condición es verdadera, se ejecuta un bloque de instrucciones; si es falsa, se ejecuta un bloque alternativo.

#### Sintaxis general:

```
Si condicion Entonces
    instrucciones si es verdadero
Sino
    instrucciones si es falso
FinSi
```

## Diagrama de Nassi-Shneiderman

Un bloque dividido verticalmente en dos subbloques: uno asociado a la condición verdadera (izquierda) y otro al caso falso o Sino (derecha). La condición se coloca en la parte superior del bloque, dividiéndolo en dos caminos posibles.



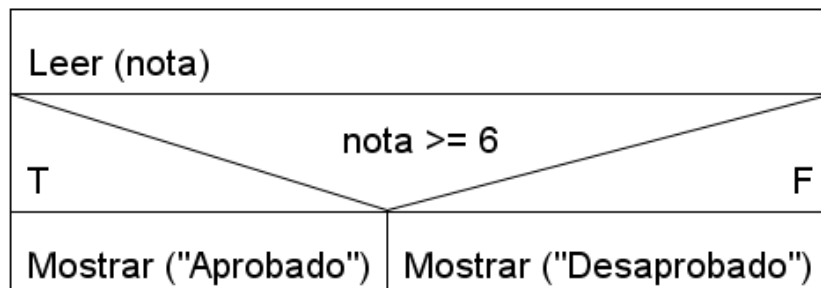
## Ejemplo Práctico

**Problema:** Determinar si un estudiante aprobó o desaprobó un examen.

### Pseudocódigo:

```
Algoritmo ResultadoExamen
    Definir nota Como Entero
    Mostrar "Ingrese la nota:"
    Leer nota
    Si nota >= 6 Entonces
        Mostrar "Aprobado"
    Sino
        Mostrar "Desaprobado"
    FinSi
FinAlgoritmo
```

### Chapin:



VAR

nota: entero

## Implementación en Python:

```
nota = int(input("Ingrese la nota: "))
if nota >= 6:
    print("Aprobado")
else:
    print("Desaprobado")
```

### 4.4. Condicional Múltiple (if-elif-else)

#### Descripción

La estructura condicional múltiple se emplea cuando existen más de dos alternativas posibles. Las condiciones se evalúan de forma secuencial: cuando una resulta verdadera, se ejecuta su bloque asociado y se sale de la estructura sin evaluar las condiciones restantes.

#### Sintaxis general:

```
Si condicion1 Entonces
    instrucciones1
SinoSi condicion2 Entonces
    instrucciones2
SinoSi condicion3 Entonces
    instrucciones3
Sino
    instrucciones por defecto
FinSi
```

#### Ejemplo Práctico

**Problema:** Clasificar el promedio de un estudiante.

#### Pseudocódigo:

```
Algoritmo ClasificarPromedio
    Definir promedio Como Real
    Escribir "Ingrese el promedio:"
    Leer promedio
    Si promedio >= 8 Entonces
        Escribir "Muy bueno"
```

```

    SinoSi promedio >= 6 Entonces
        Escribir "Bueno"
    Sino
        Escribir "Insuficiente"
    FinSi
FinAlgoritmo

```

### Implementación en Python con elif:

```

promedio = float(input("Ingrese el promedio: "))
if promedio >= 8:
    print("Muy bueno")
elif promedio >= 6:
    print("Bueno")
else:
    print("Insuficiente")

```

**Nota:** *El orden de las condiciones es crucial. Las condiciones se evalúan de arriba hacia abajo, por lo que deben ordenarse de más restrictiva a menos restrictiva.*

**Importante:** Para diseñarlo en chapin es conveniente hacer una pregunta (condición lógica) dentro de otra (se llaman Si anidados), de la siguiente manera:

| Leer (promedio)       |                   |                          |
|-----------------------|-------------------|--------------------------|
| T                     | promedio >= 8     |                          |
|                       | F                 |                          |
| Mostrar ("Muy bueno") | promedio >= 6     |                          |
|                       | T                 | F                        |
|                       | Mostrar ("Bueno") | Mostrar ("Insuficiente") |

VAR

promedio: real

## 4.5. Selector Múltiple

### Descripción

El selector multiple o case es una alternativa a los if anidados o los elif cuando se necesita comparar una variable contra múltiples valores específicos. Esta estructura es especialmente útil para menús de opciones y clasificaciones basadas en valores discretos

(es decir, un conjunto finito de valores que puede tomar una variable, y esos valores en general son un rango de números, letras o cadenas específicas).

### Ejemplo Práctico

**Problema:** Convertir un número de día de la semana (1-7) a su nombre correspondiente.

### Pseudocódigo:

```
Algoritmo case
    Definir dia Como Entero
    Leer dia
    Segun dia
        caso 1:
            Mostrar "Lunes"
        caso 2:
            Mostrar "Martes"
        caso 3:
            Mostrar "Miércoles"
        caso 4:
            Mostrar "Jueves"
        caso 5:
            Mostrar "Viernes"
        caso 6:
            Mostrar "Sábado"
        caso 7:
            Mostrar "Domingo"
        de otro modo:
            Mostrar "Día inválido"
    FinSegun
FinAlgoritmo
```

### Chapin:

VAR dia:entero

| Leer (dia)      |                  |                     |                  |                   |                  |                   |                        |
|-----------------|------------------|---------------------|------------------|-------------------|------------------|-------------------|------------------------|
| dia             |                  |                     |                  |                   |                  |                   |                        |
| 1               | 2                | 3                   | 4                | 5                 | 6                | 7                 | default                |
| Mostrar "Lunes" | Mostrar "Martes" | Mostrar "Miércoles" | Mostrar "Jueves" | Mostrar "Viernes" | Mostrar "Sábado" | Mostrar "Domingo" | Mostrar "Día inválido" |

## Implementación en Python:

Python a partir de la version 3.10 introduce la estructura match-case como alternativa moderna a las cadenas de elif cuando se necesita comparar una variable contra múltiples valores específicos.

### Sintaxis general:

```
match variable:
    case valor1:
        instrucciones1
    case valor2:
        instrucciones2
    case _:
        instrucciones_por_defecto
```

Aplicado a este problema de ejemplo quedaría:

```
dia = int(input("Ingrese número de día (1-7): "))
match dia:
    case 1:
        print("Lunes")
    case 2:
        print("Martes")
    case 3:
        print("Miércoles")
    case 4:
        print("Jueves")
    case 5:
        print("Viernes")
    case 6:
        print("Sábado")
    case 7:
        print("Domingo")
    case _:
        print("Día inválido")
```

### Ventajas del selector multiple (match-case):

- Más legible cuando hay muchos casos específicos.
- Evita repetir la variable en cada comparación.
- El caso `_` (guion bajo) captura cualquier valor no contemplado (equivalente a `else`).

**Cuándo usar match-case vs elif:** Use *match-case* para comparaciones de igualdad con valores discretos. Use *elif* para condiciones con rangos o expresiones booleanas complejas.

## 5. Estructuras Repetitivas (Iterativas)

### 5.1. Concepto

Las estructuras repetitivas (también llamadas bucles o ciclos) permiten ejecutar un conjunto de instrucciones varias veces. La repetición puede estar controlada por:

- **Un Contador o variable de control** que determina una cantidad fija de iteraciones (estructura Para).
- **Una condición lógica** que se evalúa antes de cada iteración (estructura Mientras).
- **Una condición lógica** que se evalúa después de cada iteración (estructura Repetir-Hasta).

Estas estructuras son fundamentales para procesar secuencias de datos, realizar cálculos iterativos y automatizar tareas repetitivas.

### 5.2. Estructura Para (for)

#### Descripción

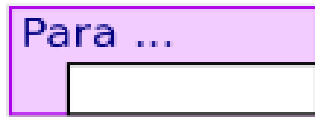
La estructura **Para** se utiliza cuando **la cantidad de iteraciones es conocida de antemano**. Emplea una variable de control (contador) que se inicializa, incrementa automáticamente en cada iteración y se compara con un valor límite.

#### Sintaxis general (pseudocódigo):

```
Para variable <- valor_inicial Hasta valor_final Hacer
    instrucciones
FinPara
```

## Diagrama de Nassi-Shneiderman

Un bloque con la especificación del rango (variable de control, valor inicial y valor final) en la parte superior, y debajo el cuerpo del ciclo que se ejecuta para cada valor de la variable.



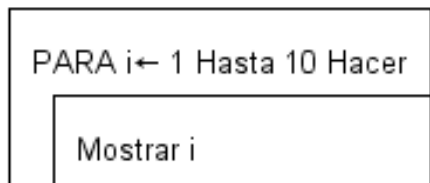
## Ejemplo Práctico

**Problema:** Mostrar los números del 1 al 10.

### Pseudocódigo:

```
Algoritmo MostrarNumeros
  Definir i Como Entero
  Para i <- 1 Hasta 10 Hacer
    Mostrar i
  FinPara
FinAlgoritmo
```

### Chapin:



VAR

i: entero

Observar que la variable **i** (llamada variable de control), se incrementa automáticamente en 1 en el ciclo PARA.

### Implementación en Python:

```
for i in range(1, 11):
    print(i)
```

**Nota sobre range():** En Python, `range(1, 11)` genera números del 1 al 10. El límite superior es exclusivo, es decir, no se toma como tal en la iteraciones, por lo tanto itera un valor menos.

### 5.3. Estructura Mientras (While)

#### Descripción

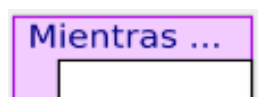
La estructura **Mientras** evalúa la condición **al inicio** del ciclo, **antes** de ejecutar el bloque de instrucciones. Si la condición es verdadera, se ejecuta el bloque y luego se vuelve a evaluar la condición. Si la condición es falsa desde el principio, el bloque puede no ejecutarse nunca. El ciclo Mientras termina con una condición con valor falso.

#### Sintaxis general:

```
Mientras condicion Hacer
    instruccion1
    instruccion2
    ...
FinMientras
```

#### Diagrama de Nassi-Shneiderman

Un bloque que contiene la condición en la parte superior y, debajo, el conjunto de instrucciones que se repite mientras la condición sea verdadera. El diagrama muestra claramente que la condición se verifica antes de cada ejecución del bloque.



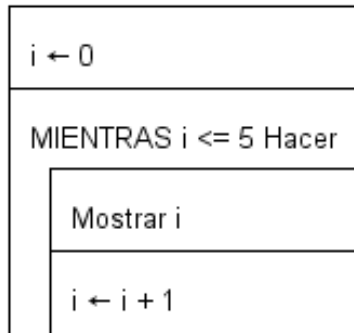
#### Ejemplo Práctico

**Problema:** Mostrar los números del 1 al 5.

#### Pseudocódigo:

```
Algoritmo ContadorAscendente
    Definir i Como Entero
    i <- 1
    Mientras i <= 5 Hacer
        Mostrar i
        i <- i + 1
    FinMientras
FinAlgoritmo
```

### Chapin:



VAR

$i$ : entero

En este caso tenemos que ir incrementando en 1 por programa la variable  $i$ .

**Advertencia:** Es fundamental que dentro del bucle exista una instrucción que modifique la condición (en este caso,  $i \leftarrow i + 1$ ), de lo contrario se creará un ciclo infinito.

### Implementación en Python:

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

**Aclaración:** La expresión  $i += 1$  es una forma abreviada en Python de escribir  $i = i + 1$  (cualquiera de las dos son válidas en el lenguaje Python).

## 5.4. Estructura Repetir-Hasta

### Descripción

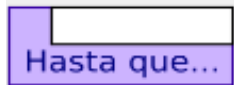
La estructura Repetir-Hasta evalúa la condición **al final** del ciclo, después de ejecutar el bloque de instrucciones. Esto garantiza que el bloque se ejecute **al menos una vez**, independientemente del valor inicial de la condición. El ciclo Repetir Hasta termina con una condición con valor verdadero.

### Sintaxis general:

```
Repetir
    instruccion1
    instruccion2
    ...
Hasta Que condicion
```

## Diagrama de Nassi-Shneiderman

Un bloque con las instrucciones en la parte superior y, al final, la condición de salida del ciclo. El diagrama enfatiza que las instrucciones se ejecutan primero y luego se evalúa la condición.



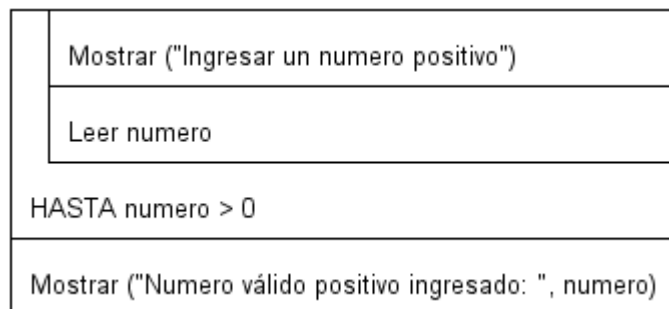
## Ejemplo Práctico

**Problema:** Solicitar un número positivo al usuario (repetir hasta que ingrese uno válido).

### Pseudocódigo:

```
Algoritmo IngresoPositivo
    Definir numero Como Entero
    Repetir
        Mostrar "Ingrese un número positivo:"
        Leer numero
    Hasta Que numero > 0
    Mostrar "Número válido ingresado:", numero
FinAlgoritmo
```

### Chapin:



VAR

numero: entero

### Implementación en Python:

```
# Python no tiene Repetir-Hasta nativo
# Se simula con while invirtiendo la condición

# primer ingreso antes del Mientras
numero = int(input("Ingrese un número positivo: "))
```

```
while numero <= 0:
    numero = int(input("Incorrecto - Ingrese un número positivo: "))
print("Número válido ingresado:", numero)
```

**Nota importante:** *Python no tiene una estructura Repetir-Hasta nativa. Para simularla, se usa while con la condición lógica invertida. Se debe realizar un ingreso previo -antes del while- de la variable de control para que la condición se evalúe correctamente en la primera iteración. No se recomienda usar while True con break, ya que no es una buena práctica de programación estructurada. En la cátedra no lo admitimos.*

## 5.5. Elección de la Estructura Repetitiva Adecuada

La elección entre las diferentes estructuras repetitivas depende de las características del problema:

- **Usar Para (for)** cuando se conoce exactamente cuántas veces se debe repetir el bloque (ejemplo: procesar exactamente 10 valores, mostrar números del 1 al 100, ...).
- **Usar Mientras (while)** cuando la repetición depende de una condición que puede ser falsa desde el inicio (ejemplo: mientras haya saldo disponible, mientras el usuario no ingrese cero).
- **Usar Repetir-Hasta** cuando se necesita garantizar al menos una ejecución del bloque (ejemplo: validación de entrada de datos donde el usuario debe ingresar al menos un valor).

La elección correcta hace que el código sea más claro, legible y fácil de mantener.

## 6. Combinación de Estructuras de Control

Es muy habitual la necesidad de utilizar más de una estructura de control combinadas para resolver un problema. Por ejemplo:

### Encontrar el mayor de N nros. y determinar el promedio de los nros. ingresados

El algoritmo para encontrar el mayor de una secuencia de números sigue estos pasos:

- Leer el primer número y guardarlo como el mayor actual.
- Leer los números restantes, comparando cada uno con el mayor actual.
- Si se encuentra un número mayor, actualizar la variable auxiliar **mayor**.
- Al finalizar el recorrido, mostrar el mayor encontrado.

El algoritmo para determinar el promedio de los nros. Ingresados sigue estos pasos:

- De cada nro. leído, se debe incrementar en 1 una variable denominada contador.
- A su vez, se debe ir acumulando (sumando) el nro. ingresado en otra variable denominada acumulador o sumador.
- Luego de finalizada la carga, se debe calcular el promedio como: acumulador/contador.

Luego, para realizar todas estas acciones bajo un mismo algoritmo, el resultado es el siguiente:

## Implementación

### Pseudocódigo:

```

Algoritmo MayoryPromediodeNnumeros
    Definir N, num, may, cont, acum Como Entero
    // Leer cantidad de nros. a procesar
    Escribir "Ingrese cantidad de nros. a procesar:"
    Leer N

    // Leer el primer número
    Escribir "Ingrese número 1:"
    Leer num

    //contar, acumular y asignar mayor para ese nro. leído
    cont <- 1
    acum <- num
    may <- num

    // Procesar los números restantes
    Para i <- 2 Hasta N Hacer
        Escribir "Ingrese número ", i, ":"
        Leer num
        cont <- cont + 1
        acum <- acum + num
        Si num > may Entonces
            may <- num
        FinSi
    FinPara
  
```

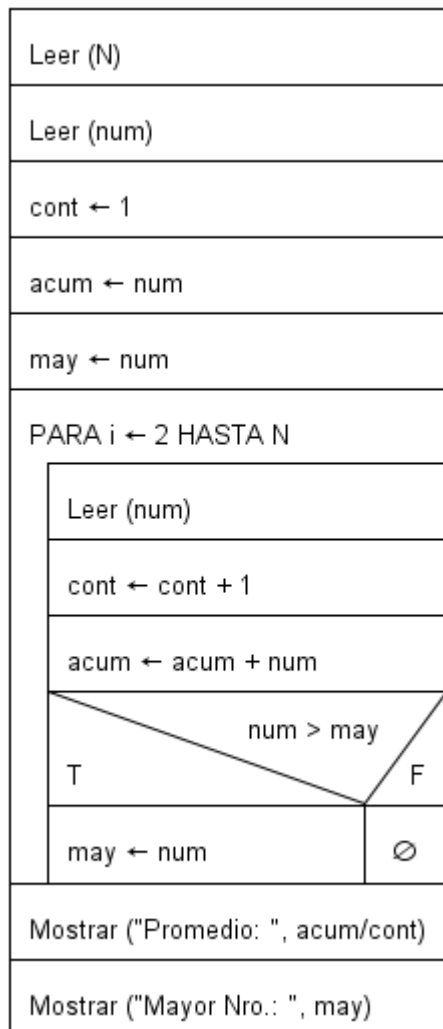
```

// Mostrar resultados
Escribir "El promedio es:", acum/cont
Escribir "El mayor es:", may

// hacemos una primera asignación de may fuera del para (al principio)
// porque si inicializamos el acumulador en 0
// y luego nos ingresan todos nros. negativos, no funcionaría el algoritmo
// porque nuestra variable may quedaría en 0.
// Este mismo método sirve para determinar el menor de una serie de nros.
// (cambiando la condición del Si).
FinAlgoritmo

```

## Chapin:



VAR

N, num, may, cont, acum: entero

## Implementación en Python:

```
# Algoritmo MayoryPromediodeNnumeros

# Leer cantidad de números a procesar
N = int(input("Ingrese cantidad de nros. a procesar: "))

# Leer el primer número
num = int(input("Ingrese número 1: "))

# contar, acumular y asignar mayor para ese número leído
cont = 1
acum = num
may = num

# Procesar los números restantes
for i in range(2, N + 1):
    num = int(input("Ingrese número" + str(i) + ": "))

    cont = cont + 1
    acum = acum + num

    if num > may:
        may = num

# Mostrar resultados
print("El promedio es: ", acum / cont)
print("El mayor es: ", may)
```

## 7. Errores Frecuentes y Buenas Prácticas

### 7.1. Errores Comunes

Se suelen cometer los siguientes errores al trabajar con estructuras de control:

- **Confundir el operador de asignación con el de comparación:** En pseudocódigo, <- es asignación y = es comparación. En Python, = es asignación y == es comparación (admitimos en la cátedra utilizar ya en los algoritmos = para asignación y == para comparación).
- **No inicializar variables de control:** En bucles while, olvidar inicializar el contador o el acumulador antes del bucle.
- **Crear ciclos infinitos:** Olvidar modificar la condición dentro del bucle Mientras, haciendo que nunca se vuelva falsa (o verdadera para el caso de Repetir Hasta).
- **Condiciones mal formuladas:** Usar operadores incorrectos.
- **Mezclar lógica de entrada/salida con la lógica del algoritmo:** No separar claramente la obtención de datos, el procesamiento y la presentación de resultados.
- **Usar la estructura incorrecta:** Emplear una estructura repetitiva cuando una condicional sería suficiente (y viceversa).
- **Usar while True con break:** Esta práctica no es recomendable en programación estructurada, ya que rompe el flujo natural del bucle. Es preferible usar una condición explícita.

### 7.2. Buenas Prácticas

- Usar nombres descriptivos para las variables.
- Indentar correctamente el código (fundamental en Python).
- Comentar el código (con #) para explicar la lógica.
- Probar casos límite (valores extremos, cero, negativos).
- Evitar condiciones complejas innecesarias.
- Diseñar antes de codificar usando diagramas chapin.

## 8. Conclusión

El dominio de las estructuras de control resulta esencial para la correcta formulación de algoritmos y constituye la base sobre la cual se apoyan las estructuras de datos y técnicas algorítmicas abordadas en unidades posteriores de la asignatura.

La capacidad de combinar apropiadamente estructuras secuenciales, condicionales y repetitivas permite al estudiante resolver problemas de complejidad creciente, siguiendo siempre los principios de la programación estructurada: claridad, modularidad y verificabilidad.

Se recomienda a los estudiantes practicar intensivamente con los ejercicios propuestos en las guías, experimentar con variaciones de los ejemplos presentados y, fundamentalmente, desarrollar el hábito de planificar la solución mediante diagramas chapin antes de implementar en código.

## 9. Recursos Adicionales

Para profundizar en los temas tratados, se sugieren los siguientes recursos:

- Joyanes Aguilar, L. (2008). Fundamentos de Programación: Algoritmos, estructura de datos y objetos. McGraw-Hill.
- Cairó, O. (2005). Metodología de la Programación: Algoritmos, diagramas de flujo y programas. Alfaomega.
- Documentación oficial de Python: <https://docs.python.org/es/>